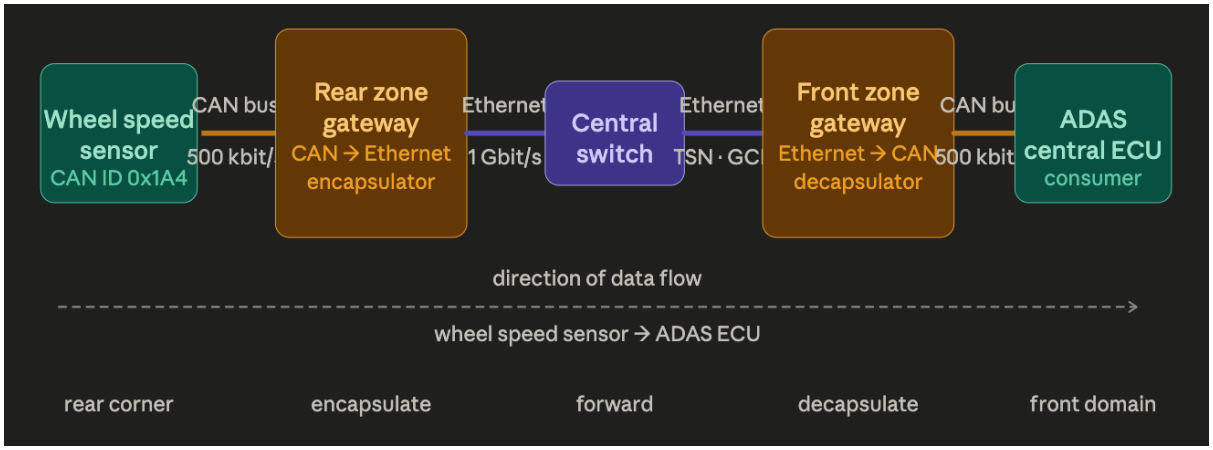
End-to-end story of a single CAN message making its journey.

The scenario

A wheel speed sensor on the rear-left corner of a vehicle sends a CAN message every 10 ms. The sensor is on a local CAN bus connected to the rear zone ECU (Electronic Control Unit). The ADAS (Advanced Driver Assistance System) central computer — sitting at the front of the vehicle — needs that wheel speed value to run its stability control algorithm. The two ECUs are connected by an Ethernet backbone, not a direct CAN wire.

The question is: ***how does a CAN message that was born on a CAN bus in the rear of the car arrive on a CAN bus in the front of the car, crossing an Ethernet backbone in between?***



There are two gateways. Each gateway has one foot on the CAN bus and one foot on the Ethernet backbone.

The rear gateway is the **encapsulator** — it picks up CAN frames from the sensor and wraps them for Ethernet transport.

The front gateway is the **decapsulator** — it unwraps the Ethernet packet and puts the CAN frame back onto a local CAN bus where the ADAS ECU can read it as if the sensor were directly wired.

STEP 1 — ON THE CAN BUS

The wheel speed sensor transmits a CAN frame

The sensor broadcasts onto the CAN bus every 10 ms. The frame contains:

Identifier: `0x1A4` — this is the message ID that every node on the CAN bus uses to know what the message means
Data: `0x03 0xE8` — two bytes encoding 1000 RPM wheel speed
DLC: `2` — two data bytes

The rear zone gateway's CAN controller is listening. It receives the frame off the bus exactly as any other CAN node would. At the moment the Start-of-Frame bit arrives, the gateway's gPTP hardware latches the arrival time: `T_arrival = 5,000,000,520 ns` (just over 5 seconds since network boot, in gPTP time).

STEP 2 — INSIDE THE REAR ZONE GATEWAY

The gateway decides: should this frame be tunnelled?

Not every CAN frame on this bus needs to cross to the front domain. The gateway has a routing table — a list of CAN IDs that are configured to be tunnelled over Ethernet, and which AVTP stream each one maps to.

CAN ID `0x1A4` is in the table: it maps to stream `stream_id = AA:BB:CC:11:22:33 : 0x0003`, destined for the front zone gateway.

CAN frames with other IDs — say local door control messages — are not in the table and stay on the local CAN bus. They never touch the Ethernet backbone.

STEP 3 — INSIDE THE REAR ZONE GATEWAY

The gateway wraps the CAN frame in ACF and AVTP

The gateway software builds a packet. Think of it as two envelopes, one inside the other.

Inner envelope — the ACF message:
"This is a CAN frame. It came from bus 2. Its identifier is 0x1A4. It is a classical CAN frame (not FD). Its data is 0x03 0xE8. It arrived at gPTP time 5,000,000,520 ns."

Outer envelope — the AVTP stream header:
"This packet belongs to stream AA:BB:CC:11:22:33:0003. It is sequence number 147. Please act on the enclosed payload at gPTP time 5,000,002,020 ns." — that presentation timestamp is T_arrival + 1,500 ns, a pre-configured transit offset that accounts for the expected Ethernet crossing time plus the front gateway's playout buffer depth.

The whole thing is then given an Ethernet header with EtherType `0x22F0` (the registered AVTP EtherType) and a VLAN tag with PCP=3 to place it in the right traffic class.

STEP 4 — ON THE ETHERNET BACKBONE

The Ethernet frame crosses the backbone like any other TSN stream

The Ethernet frame enters the central switch. The switch does not know or care that there is a CAN frame inside it. It sees:

— EtherType `0x22F0` → this is an AVTP stream
— VID and PCP → this frame belongs to the diagnostics VLAN, priority class 3
— Destination MAC → forward out port 4 toward the front zone gateway

The GCL (Gate Control List) in the switch has a gate window allocated for this stream's traffic class. The frame waits at most one GCL cycle (say 1 ms) before its gate opens and it exits the switch. The TSN machinery guarantees bounded latency — the frame will reach the front gateway within a known worst-case time.

The CAN frame inside is completely invisible to the switch. From the switch's perspective this is just another Ethernet frame in queue 3.

STEP 5 — INSIDE THE FRONT ZONE GATEWAY

The front gateway receives and decapsulates

The front gateway's NIC receives the Ethernet frame. Its AVTP parser reads the stream header:

— `stream_id = AA:BB:CC:11:22:33:0003` → this is the wheel speed tunnel stream, I know this one
— `seq_num = 147` → last one I received was 146, no gap, no lost frames
— `avtp_timestamp = 5,000,002,020 ns` → this is when I must transmit the CAN frame

The ACF parser then unwraps the inner envelope and recovers the CAN frame exactly: ID `0x1A4`, data `0x03 0xE8`, DLC 2, classical CAN, bus_ID 2 → route to CAN port 2.

The gateway places this reconstructed CAN frame into its playout buffer, keyed by the presentation timestamp `5,000,002,020 ns`.

STEP 6 — BACK ONTO A CAN BUS

The CAN frame is transmitted at the scheduled time

The front gateway's gPTP clock ticks forward. At the instant it reaches `5,000,002,020 ns`, the playout buffer fires. The gateway's CAN controller transmits the frame onto its local CAN bus.

The ADAS ECU, which is listening on that CAN bus, receives a frame with identifier `0x1A4` containing the value `0x03 0xE8`.

The ADAS ECU has no idea this frame came from across an Ethernet backbone. It looks exactly like a CAN frame from a wheel speed sensor directly wired to its local CAN bus. The identifier is the same. The data bytes are the same. The timing is deterministic and bounded.

The CAN tunnel is transparent. The Ethernet backbone is invisible to both the sensor and the ADAS consumer.

The three things that make this work — in plain terms

Identity preservation. The CAN identifier 0x1A4, the data bytes, and all the flag bits travel inside the ACF message untouched. When the front gateway reconstructs the CAN frame it is a byte-for-byte copy of the original. No translation, no protocol conversion, no semantic reinterpretation.

Timing preservation. The AVTP presentation timestamp is computed so that the far-end CAN frame appears on the far bus at a predictable, bounded time after the original was sent. Because both gateways share a common gPTP clock synchronised across the Ethernet backbone, the front gateway can schedule its CAN transmission with nanosecond-level precision relative to the original event.

Transparency. Neither the wheel speed sensor nor the ADAS ECU is modified or even aware that an Ethernet backbone exists between them. The gateways do all the work. Legacy CAN nodes need no Ethernet capability whatsoever.